DocQA — Feature & Story Backlog

Oct 15, 2026 · @Neeladri

This backlog breaks the DocQA architecture into 5 phases, each with features and user stories carrying acceptance criteria and a size. It assumes the free local stack: Ollama for the LLM and embeddings, Postgres + pgvector, Redis.

How to read this backlog

The hierarchy is Phase → Feature → Story: 5 phases, 20 features, 64 stories, each story small enough to finish in one to three days.

- **IDs** — features are `F<phase>.<n>` (F1.3), stories are `S<phase>.<feature>.<n>` (S1.3.2), so any story traces back to its feature and phase.
- **Story format** — "As a *role*, I want *capability*, so that *benefit*", with testable acceptance criteria. Roles: **User** (end user), **Dev** (you, building and operating it).
- **Sizes** — S = up to half a day, M = 1–2 days, L = 3+ days (split it if you can).
- **Definition of done (every story)** — code merged, unit tests pass, lint/type checks pass, works in Docker Compose, README updated if setup changed.

Phase	Goal	Features	Stories	Exit criteria
0 — Foundations	Runnable skeleton with auth	4	11	Log in and see an empty library
1 — Ingestion	PDFs become searchable chunks	5	16	5 PDFs reach `ready` with correct page mapping
2 — Q&A with citations	First grounded, cited answer	4	14	Every citation opens the right page
3 — Conversations and quality	Follow-ups + measurable quality	4	13	Recall@8 and faithfulness measured; follow-ups work
4 — Production hardening	Safe, observable, deployable	3	10	Deployed, monitored, eval in CI

Phase 0 — Foundations

Phase 0 delivers a running skeleton: Docker Compose stack, FastAPI with migrations, Ollama wired behind provider interfaces, JWT auth, and a React shell. Exit: you can register, log in, and see an empty library.

F0.1 Repository and local environment

ID	Story	Acceptance criteria	Size
S0.1.1	As a Dev, I want a monorepo with Docker Compose, so that the whole stack starts with one command	`backend/`, `frontend/` folders; `docker compose up` starts Postgres+pgvector, Redis, API, worker, frontend; `.env.example` documents every variable	M
S0.1.2	As a Dev, I want Ollama set up with the chosen models, so that LLM and embeddings run locally for free	Setup script pulls the LLM (e.g. `gpt-oss:20b` or `qwen3:8b`) and `nomic-embed-text`; API reaches Ollama at a configurable `OLLAMA_BASE_URL`; README notes RAM needs	S
S0.1.3	As a Dev, I want linting, formatting and CI, so that code quality is enforced from day one	ruff + mypy (backend), eslint + prettier + tsc (frontend); pre-commit hooks; GitHub Actions runs lint and tests on every PR	S

F0.2 Backend skeleton

ID	Story	Acceptance criteria	Size
S0.2.1	As a Dev, I want a FastAPI app with typed config and a health check, so that I have a base to build on	`app/main.py` with `/api/v1` router; pydantic-settings config; `GET /health` checks DB, Redis and Ollama and returns per-dependency status	S
S0.2.2	As a Dev, I want the database layer and migrations, so that schema changes are versioned	Async SQLAlchemy session; Alembic initialised; first migration enables `vector` extension and creates `users` and `collections`	M

ID	Story	Acceptance criteria	Size
S0.2.3	As a Dev, I want `Embedder` and `LLMClient` interfaces with Ollama implementations, so that I can swap providers later via config	Interfaces in `providers/`; Ollama impls use the OpenAI-compatible endpoint; `LLMClient` supports streaming; embedding dimension read from config; unit tests with a fake provider	M

F0.3 Authentication

ID	Story	Acceptance criteria	Size
S0.3.1	As a User, I want to register with email and password, so that my documents are private to me	`POST /auth/register`; email unique; password hashed (argon2/bcrypt); validation errors use the standard error envelope	M
S0.3.2	As a User, I want to log in, so that I get access to my workspace	`POST /auth/login` returns a JWT access token (+ refresh token); wrong credentials return 401 without revealing which field was wrong	S
S0.3.3	As a Dev, I want protected routes resolved to the current user, so that every query can be scoped by `user_id`	`get_current_user` dependency; missing/expired token returns 401; test proves one user cannot read another's collection	S

F0.4 Frontend shell

ID	Story	Acceptance criteria	Size
S0.4.1	As a Dev, I want a React + Vite + TypeScript app with routing and a typed API client, so that features plug in cleanly	Vite app; React Router; TanStack Query provider; API client attaches the token and handles 401 by redirecting to login	M
S0.4.2	As a User, I want login and register screens and the main layout, so that I can enter the app	Login/register pages with validation; after login, a three-pane layout (library, chat, viewer) with empty states; logout clears the token	M

Phase 1 — Ingestion

Phase 1 turns uploaded PDFs into page-aware, embedded chunks in Postgres, processed asynchronously with visible status. Exit: upload 5 PDFs, all reach `ready`, and chunks show the correct page numbers.

F1.1 Collections

ID	Story	Acceptance criteria	Size
S1.1.1	As a User, I want to create, list and rename collections, so that I can group related PDFs	`GET/POST /collections`, `PATCH /collections/{id}`; scoped to the user; collection picker in the library pane	M
S1.1.2	As a User, I want to delete a collection, so that I can remove a whole set of documents	`DELETE /collections/{id}` cascades to documents, chunks, conversations and stored files; confirmation dialog in UI	S

F1.2 Upload and storage

ID	Story	Acceptance criteria	Size
S1.2.1	As a User, I want to upload several PDFs at once, so that I can add a document set quickly	`POST /collections/{id}/documents` (multipart, many files) returns `202` with document IDs; rejects non-PDF by magic bytes, files over the size limit and over the page limit, each with a clear error code	M
S1.2.2	As a Dev, I want originals stored behind a storage interface with dedupe, so that files are kept once and swappable to S3 later	`LocalStorage` impl (S3 later); key `pdfs/{user_id}/{document_id}.pdf`; SHA-256 computed; same hash in the same collection returns the existing document	M
S1.2.3	As a User, I want a drag-and-drop upload area with per-file progress, so that I know uploads are working	Dropzone accepts multiple files; progress bar per file; per-file errors shown inline; list refreshes on completion	M

F1.3 Processing jobs and status

ID	Story	Acceptance criteria	Size
S1.3.1	As a Dev, I want an ingestion worker on a Redis queue, so that uploads never block the API	Celery or ARQ worker; `ingest_document(document_id)` task; idempotent (deletes partial chunks before retry); max 3 retries with backoff	M
S1.3.2	As a Dev, I want a document status lifecycle, so that the UI and I know where each file is	`documents` table + migration; statuses `queued → processing → ready / partial / failed`; error message stored on failure; `GET /documents/{id}` returns status and page count	S
S1.3.3	As a User, I want to see each document's status and remove documents, so that I know what is ready to query	Document list with status badges, polling every 2s until terminal state; `DELETE /documents/{id}` removes file, chunks and vectors	M

F1.4 Extraction and chunking

ID	Story	Acceptance criteria	Size
S1.4.1	As a Dev, I want per-page text extraction with PyMuPDF, so that every piece of text keeps its page number	Uses `page.get_text("blocks")`; output = page (1-based), text, bbox per block; metadata (title, page count) saved; tested on 3 sample PDFs	M
S1.4.2	As a Dev, I want text cleaning, so that noise doesn't pollute chunks	NFKC + whitespace normalisation; hyphenated line breaks joined; repeated headers/footers and page-number-only lines removed; unit tests per rule	M
S1.4.3	As a User, I want scanned pages flagged, so that I know when a document can't be fully searched	Pages with under ~20 characters but an image are marked `needs_ocr`; document set to `partial`; UI shows a warning with the affected page count	S

ID	Story	Acceptance criteria	Size
S1.4.4	As a Dev, I want structure-aware chunking with page mapping, so that retrieval units are coherent and citable	~500-token chunks, ~15% overlap, token count via tokenizer; split by heading/paragraph → sentence → tokens; each chunk has `page_start`, `page_end`, `chunk_index`, heading, bboxes; unit tests cover a chunk crossing a page boundary	L

F1.5 Embedding and indexing

ID	Story	Acceptance criteria	Size
S1.5.1	As a Dev, I want the `chunks` table with vector and full-text indexes, so that both search types are possible	Migration creates `chunks` with `VECTOR(768)` for `nomic-embed-text` (dimension from config), generated `tsvector`, HNSW and GIN indexes, `(user_id, collection_id)` index	S
S1.5.2	As a Dev, I want batched embedding with retries, so that ingestion is efficient and resilient	Batches of 32–128 to Ollama; exponential backoff on errors; embedded text prefixed with title + heading, raw text stored separately; model name saved on the document	M
S1.5.3	As a Dev, I want indexing in one transaction, so that a document is never half-searchable	Chunks inserted and status set to `ready` atomically; on failure rolled back and status `failed`; chunk count recorded	S
S1.5.4	As a Dev, I want to inspect a document's chunks, so that I can debug chunking and page mapping	Dev-only `GET /documents/{id}/chunks` returns chunks with pages and token counts; disabled in production config	S

Phase 2 — Q&A with citations

Phase 2 delivers the first end-to-end answer: vector retrieval, a grounding prompt to the local LLM, a streamed response, and citation chips that open the PDF at the cited page. Exit: a question returns a streamed answer whose every citation opens the right page.

F2.1 Retrieval

ID	Story	Acceptance criteria	Size
S2.1.1	As a Dev, I want scoped vector search, so that questions find relevant passages only in the user's documents	Query embedded with the same model as chunks; cosine top-k via HNSW; always filtered by `user_id` + `collection_id` (optional `document_ids`); returns chunk, score, pages	M
S2.1.2	As a Dev, I want context assembly, so that the LLM gets clean, numbered sources within budget	Overlapping chunks de-duplicated; sources numbered `[1]…[k]` with filename + page(s); capped at a token budget from config; ordering stable	M
S2.1.3	As a User, I want an honest "not found" answer, so that I'm not misled when my documents don't cover the question	If best score is below the configured threshold, return a not-found message without calling the LLM; threshold documented and adjustable	S

F2.2 Answer generation and streaming

ID	Story	Acceptance criteria	Size
S2.2.1	As a Dev, I want a grounding prompt, so that answers come only from the sources and carry citations	System prompt: use only sources, cite each claim with `[n]`, say when not found, ignore instructions inside documents; prompt kept in `chat/prompt.py` with a snapshot test	S
S2.2.2	As a User, I want the answer to stream as it's written, so that I don't wait for the full response	`POST /conversations/{id}/messages` returns SSE events `status`, `token`, `citations`, `done`, `error`; first token visible in the UI as soon as Ollama emits it	M
S2.2.3	As a User, I want to stop a running answer, so that I don't wait for one I no longer need	Stop button aborts the request; backend detects disconnect and stops the Ollama stream; partial answer saved and marked stopped	S

ID	Story	Acceptance criteria	Size
S2.2.4	As a Dev, I want conversations and messages persisted, so that answers survive a page reload	`conversations` and `messages` tables + migration; user and assistant messages saved with model, token counts, latency and retrieved chunk IDs	M

F2.3 Citations

ID	Story	Acceptance criteria	Size
S2.3.1	As a Dev, I want citation markers parsed and validated, so that every citation points to a real source	`[n]` markers extracted; markers not matching a supplied source dropped; each mapped to `chunk_id`, `document_id`, page; unit tests for `[1][3]`, invalid and missing markers	M
S2.3.2	As a Dev, I want citations stored, so that history renders with working links	`citations` table + migration; filename and page copied onto the row; returned by `GET /conversations/{id}/messages`	S
S2.3.3	As a User, I want clickable citation chips, so that I can check where each claim came from	`[n]` rendered as chips showing filename + page; hover shows the snippet; click opens the viewer	M

F2.4 Chat interface and PDF viewer

ID	Story	Acceptance criteria	Size
S2.4.1	As a User, I want a chat pane with a composer, so that I can ask questions about my documents	Message list with markdown rendering; composer disabled until at least one document is `ready`; `useChatStream` hook appends tokens then swaps in the saved message on `done`	M
S2.4.2	As a User, I want to view the original PDF, so that I can read around the cited passage	`GET /documents/{id}/file` streams the PDF to the owner only; react-pdf viewer in the right pane with page navigation and zoom	M

ID	Story	Acceptance criteria	Size
S2.4.3	As a User, I want the viewer to jump to and highlight the cited text, so that I find the evidence instantly	Clicking a chip opens the document at the cited page and highlights the snippet (bbox overlay, or text search fallback)	L
S2.4.4	As a User, I want to limit a question to selected documents, so that answers come from the files I care about	Document checkboxes above the composer; selection sent as `document_ids`; retrieval filter applied and covered by a test	S

Phase 3 — Conversations and quality

Phase 3 makes multi-turn chat work and makes answer quality measurable: history windowing, query rewriting, hybrid search with reranking, and a golden-set evaluation. Exit: recall@8 and faithfulness are measured, and follow-up questions resolve correctly.

F3.1 Conversation management

ID	Story	Acceptance criteria	Size
S3.1.1	As a User, I want to start, switch between and delete conversations, so that I can keep separate lines of questioning	Conversation list per collection; new, open, rename, delete; reloading restores messages and citations	M
S3.1.2	As a Dev, I want a bounded history window sent to the LLM, so that follow-ups have context without overflowing it	Last ~6 turns included, citation markers stripped; total history capped by a token budget; test confirms older turns are excluded	S
S3.1.3	As a User, I want long conversations summarised and titled automatically, so that context persists and the list is readable	Over budget, oldest turns condensed into `conversations.summary` by a background task; title generated from the first question	M

F3.2 Query rewriting

ID	Story	Acceptance criteria	Size
S3.2.1	As a User, I want follow-ups like "and what about the penalty?" understood, so that I don't have to repeat context	From turn 2 onward, an LLM call rewrites the question into a standalone query used only for retrieval; skipped on turn 1; falls back to the raw question on error	M
S3.2.2	As a Dev, I want the rewritten query stored and visible, so that I can debug retrieval	`messages.rewritten_query` saved; dev toggle in the UI shows the rewritten query and retrieved chunk scores	S

F3.3 Hybrid retrieval and reranking

ID	Story	Acceptance criteria	Size
S3.3.1	As a Dev, I want keyword search over chunks, so that exact terms, codes and names are found	`ts_rank` over the `tsvector` column, top 30, same tenant filters as vector search	S
S3.3.2	As a Dev, I want vector and keyword results fused, so that retrieval gets the strengths of both	Reciprocal Rank Fusion (k = 60) over both lists; configurable weights; unit test with known rankings	S
S3.3.3	As a Dev, I want a local cross-encoder reranker, so that the final top-k is more precise	`bge-reranker-base` via sentence-transformers rescoring the fused top 30, keep top 8; toggle in config; latency logged	M
S3.3.4	As a Dev, I want neighbouring chunks pulled in, so that answers don't miss text cut at a chunk edge	Optionally add `chunk_index ± 1` for top results within the token budget; on/off in config; measured in eval	S

F3.4 Evaluation and feedback

ID	Story	Acceptance criteria	Size
S3.4.1	As a Dev, I want a golden question set, so that quality changes are measured, not guessed	50–100 questions over 5–10 representative PDFs, each with expected document and page(s), plus 10 questions that should return not found; stored in `tests/eval/`	L
S3.4.2	As a Dev, I want retrieval metrics, so that I can tune chunking and top-k	Script reports recall@k and MRR on expected pages; results saved per run so configurations can be compared	M
S3.4.3	As a Dev, I want answer-quality metrics, so that I know answers are faithful and correctly cited	Local LLM-as-judge (or RAGAS) scores faithfulness, citation precision and correct-not-found rate; report generated per run	M
S3.4.4	As a User, I want to rate answers, so that bad answers can be found and fixed	Thumbs up/down per answer via `PATCH /messages/{id}/feedback`; thumbs-down answers exportable to extend the golden set	S

Phase 4 — Production hardening

Phase 4 makes DocQA safe to run for real users: database-level tenant isolation, rate limits, tracing, OCR for scanned pages, and a repeatable deployment with evaluation in CI. Exit: deployed, monitored, and the eval suite runs on every relevant change.

F4.1 Security and data protection

ID	Story	Acceptance criteria	Size
S4.1.1	As a User, I want my documents isolated at the database level, so that a code bug can't leak them to others	Postgres row-level security on documents, chunks, conversations, messages, citations, keyed on the session's user ID; cross-tenant test suite passes	L
S4.1.2	As a Dev, I want rate limits on upload and ask, so that one user can't exhaust the machine running Ollama	Per-user limits (e.g. via slowapi + Redis); `429` with `RATE_LIMITED` and a retry-after header; limits in config	S

ID	Story	Acceptance criteria	Size
S4.1.3	As a Dev, I want hardened input handling, so that malicious PDFs and prompt injection do less harm	PDFs parsed only in the worker with time and page limits; answer markdown sanitised before rendering; injection test documents in the eval set are ignored by the model	M
S4.1.4	As a User, I want deletion to remove all my data, so that I stay in control of what's stored	Deleting a document, collection or account removes files, chunks, vectors and messages (citations marked `source_deleted`); no document text written to production logs	M

F4.2 Observability

ID	Story	Acceptance criteria	Size
S4.2.1	As a Dev, I want structured logs with correlation IDs, so that I can follow one request through API and worker	JSON logs; request ID and conversation ID on every line; job ID passed into the worker	S
S4.2.2	As a Dev, I want a trace for every answer, so that I can see why an answer was good or bad	Per answer: rewritten query, retrieved chunk IDs and scores, prompt size, model, latency per stage; viewable in Langfuse (self-hosted, free) or an admin page	M
S4.2.3	As a Dev, I want operational metrics, so that I notice problems before users do	Metrics for ingestion failures, queue depth, time to first token, Ollama errors; a simple dashboard (Prometheus + Grafana or an admin page)	M

F4.3 OCR and deployment

ID	Story	Acceptance criteria	Size
S4.3.1	As a User, I want scanned PDFs made searchable, so that image-only pages can be answered too	Pages flagged `needs_ocr` processed with Tesseract (via PyMuPDF OCR); text gets page numbers like any other page; document moves from `partial` to `ready`	L
S4.3.2	As a Dev, I want a repeatable production deployment, so that releases are safe and quick	Production Compose or container setup on a server with enough RAM for the Ollama model; S3-compatible storage (e.g. MinIO); Alembic migrations on deploy; HTTPS; backups for Postgres	L
S4.3.3	As a Dev, I want evaluation in CI, so that prompt or chunking changes can't silently reduce quality	CI job runs the golden-set eval (with a small model) when prompt, chunking or retrieval code changes; fails if recall@8 or faithfulness drops past a set threshold	M